

SmartRAG-Simple

RAG(Retrieval-Augmented Generation, 检索增强生成)

极简可演示的RAG

5 步理解核心流程

完整代码

```
# ===== 第 1 步: 准备知识库 (3 条文档) =====
from langchain_core.documents import Document

docs = [
    Document(page_content="RAG 是检索增强生成, 它结合了信息检索和大语言模型。"),
    Document(page_content="RAG 的核心步骤: 加载文档 → 切分 → 向量化 → 检索 → 生成。"),
    Document(page_content="向量数据库 (如 FAISS) 用于高效存储和检索文档向量。")
]

# ===== 第 2 步: 向量化并构建索引 =====
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = FAISS.from_documents(docs, embeddings)

# ===== 第 3 步: 检索: 根据问题找到最相关的文档 =====
retriever = vectorstore.as_retriever(search_kwargs={"k": 2}) # 返回最相关的 2 条

question = "RAG 有哪些步骤?"
retrieved_docs = retriever.invoke(question)

print("📄 检索到的相关文档: ")
for i, doc in enumerate(retrieved_docs, 1):
    print(f"    {i}. {doc.page_content}")

# ===== 第 4 步: 生成: 基于检索结果生成答案 =====
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 为了完全本地运行, 这里使用一个极简的"伪生成器"来演示原理
# (实际项目中可替换为任何 LLM, 如 Ollama、GPT4All 等)
def simple_generator(context, question):
    # 这个函数模拟 LLM 的生成过程: 将上下文和问题组合成答案
    return f"根据知识库, 我找到以下信息: \n{context}\n\n基于这些信息, 回答你的问题: {question}"

# 构建上下文
context = "\n".join([doc.page_content for doc in retrieved_docs])

# ===== 第 5 步: 输出最终答案 =====
answer = simple_generator(context, question)
print("\n🗨️ 生成的答案: ")
print(answer)
```

运行结果示例

当你运行这段代码时, 会看到类似以下的输出:



检索到的相关文档：

1. RAG 的核心步骤：加载文档 → 切分 → 向量化 → 检索 → 生成。
2. RAG 是检索增强生成，它结合了信息检索和大语言模型。



生成的答案：

根据知识库，我找到以下信息：

RAG 的核心步骤：加载文档 → 切分 → 向量化 → 检索 → 生成。

RAG 是检索增强生成，它结合了信息检索和大语言模型。

基于这些信息，回答你的问题：RAG 有哪些步骤？

简单的说明

步骤	对应 RAG 核心环节	关键操作
第 1 步	文档加载	准备知识库数据（可替换为 <code>PyPDFLoader</code> 加载真实 PDF）
第 2 步	向量化 + 索引构建	使用 HuggingFace 本地模型 + FAISS 向量库
第 3 步	检索	<code>retriever.invoke(question)</code> 找到最相关的 2 条文档
第 4-5 步	生成	将检索到的上下文 + 用户问题组合成最终答案

扩展说明

- 替换为真实 PDF：将第 1 步替换为 `PyPDFLoader`（详见文档前面的"处理 PDF 文件的文档加载器"章节）。
- 替换为真正的 LLM：将 `simple_generator` 函数替换为 `ChatOpenAI`、`ChatOllama` 或 `HuggingFaceHub`。
- 调整检索精度：修改 `k` 值（返回文档数量）或切换到进阶检索策略（如混合检索、重排序）。

在此基础上逐步叠加文档中的其他高级特性（如分块策略、多轮对话、提示词模板等）。

处理 PDF 文件的文档加载器

```
# 1. 从 langchain_community 导入 PDF 加载器
from langchain_community.document_loaders import PyPDFLoader

# 2. 创建加载器实例，指定 PDF 文件路径
loader = PyPDFLoader("example.pdf") # 请替换为你的文件路径

# 3. 加载文档：将 PDF 内容读取为 LangChain 的 Document 对象列表
documents = loader.load()

# 4. （可选）查看文档数量，验证加载结果
print(f"成功加载 {len(documents)} 页内容。")

# 5. （可选）打印第一页的部分内容，查看提取的文本
print(documents[0].page_content[:100]) # 打印前100个字符
```

分块策略

字符分块（Character Splitting）

简单使用如下：

```
from langchain.text_splitter import CharacterTextSplitter # 导入字符分块器

# 创建分块器：每块100字符，重叠20字符
splitter = CharacterTextSplitter(chunk_size=100, chunk_overlap=20)

# 待分块的长文本
text = "这是一个长文本。它将被切成小块。每块有固定长度，便于后续处理。"

# 执行分块，返回文本块列表
chunks = splitter.split_text(text)

# 打印结果
print(chunks)
```

参考结果：

```
[ '这是一个长文本。它将被切成小块。每块有固定长', '每块有固定长度，便于后续处理。' ]
```

定义：什么是字符分块？

字符分块是一种基于文本长度的、最简单的文档拆分策略。它的核心目标是**将长文本切割成许多符合特定大小（如字符数）的小块**，以便于后续处理，例如输入给有上下文窗口限制的大语言模型。

这种方法的指导思想非常朴素：不考虑文本的语义或结构，只关注文本的物理长度。

基本原理：如何工作？

它的工作原理可以概括为以下几个步骤，非常简单直观：

- 设定分隔符**：首先，你需要指定一个分隔符（**Separator**），比如一个句号（`.`）、一个空格（）或一个换行符（`\n`）。文本将主要依据这个分隔符来被切分。
- 设定块大小与重叠**：你需要定义块大小（`chunk_size`），即每个小块最多包含多少个字符；以及块重叠（`chunk_overlap`），即相邻两个块之间共享多少个字符，这有助于保持上下文的连贯性。
- 执行切割**：算法会尝试在你指定的分隔符处进行切割，并尽量让每个块的长度不超过设定的 `chunk_size`。
- 处理特殊情况**：如果在 `chunk_size` 范围内找不到指定的分隔符，这个块可能会被强制截断，导致其长度超过设定值。

优缺点与适用场景

- 优点**：实现简单，易于理解和使用，能快速生成大小一致的文本块。
- 缺点**：完全无法识别文本的语义结构，可能会将一个完整的句子或段落切断，导致信息碎片化，破坏上下文。
- 适用场景**：适用于对文本分割要求不高的简单场景，或作为了解分块技术的入门实践。

学术文献与参考来源

官方实现

这是 LangChain 库中的一个基础工具，你可以直接查阅其官方文档和源代码。

- 官方文档**：LangChain 的官方文档对 `CharacterTextSplitter` 有明确定义，称其为“**基于长度的文本分割器**”，并详细解释了其基于 `separator` 和 `chunk_size` 的工作原理。
- 源代码**：其实现逻辑（例如如何使用正则表达式进行切割）可以在 LangChain 的 GitHub 代码库中找到。

相关学术研究

虽然“字符分块”本身是一项工程实践，但“文本分割”（Text Splitting）或“句子分割”（Sentence Splitting）是NLP领域的经典研究课题。将文本切分为语义完整、大小合适的单元，是提升信息检索和模型生成效果的关键。学术界的相关研究主要关注更智能的分割方法，可以看作是字符分块复杂化、智能化的演进。

- 经典数据集**：用于训练和评估分割模型的数据集，如 **WebSplit**、**WikiSplit** 和 **BiSECT**。
- 前沿方法**：研究重点已从早期的**基于语法规则**的方法，转向了**基于语义和深度学习**的方法，例如使用 Transformer 架构的模型。

递归字符分块（Recursive Character Splitting）

递归字符文本分割器（RecursiveCharacterTextSplitter）是 LangChain 官方推荐的文本分割器，它在保持文本语义完整性和控制块大小之间提供了很好的平衡。

💡 定义与基本原理

递归字符分割器的设计目标是尽可能保持文本的自然语义边界（如段落、句子）不被切断。

它的核心原理是维护一个**从粗到细**的分隔符优先级列表，并逐级尝试对文本进行切分。

- 设定分隔符层级**：默认的分隔符列表是 `["\n\n", "\n", " ", ""]`，分别对应**段落**、**换行**、**空格**和**字符**。
- 递归尝试切分**：
 - 首先，它尝试用最粗粒度的分隔符（`\n\n`，即段落边界）来切分文本。
 - 如果切分后的某个部分仍然超过设定的 `chunk_size`（块大小），它会使用下一个分隔符（`\n`，即换行符）来进一步切分这个超长的部分。
 - 这个过程会持续进行，直到文本块的大小符合要求为止。
- 兜底策略**：如果到了最后一级分隔符 `" "`（即逐字符），文本块仍然过长，分割器才会在字符级别进行强制截断。

🐍 简单的 Python 示例

这个例子展示了如何创建一个递归字符分割器，并用于切分一段长文本。

```
from langchain_text_splitters import RecursiveCharacterTextSplitter # 导入分割器

# 1. 创建分割器实例
# chunk_size: 每个块的最大字符数
# chunk_overlap: 相邻块之间重叠的字符数, 用于保持上下文连贯性
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=100, # 每块最多100个字符
    chunk_overlap=20 # 块与块之间重叠20个字符
)

# 2. 准备一段长文本
long_text = """这是第一段。它包含一个完整的句子。
这是第二段。它同样是一个完整的段落。
这是第三段。它可能包含多个句子。这段文字会被我们的分割器处理。"""

# 3. 执行切分
chunks = text_splitter.split_text(long_text)

# 4. 查看结果
for i, chunk in enumerate(chunks):
    print(f"块 {i+1}: {chunk}\n")
```

可参考的文献与出处

核心框架与工程参考

- **官方文档与API参考**: LangChain官方文档详细介绍了 `RecursiveCharacterTextSplitter` 的设计理念和用法, 其中明确指出“对于大多数用例, 建议从 `RecursiveCharacterTextSplitter` 开始”。其Python API参考提供了完整的参数说明 (如 `separators`, `keep_separator` 等)。
- **开源实现**: LangChain的源代码是该分割器最权威的实现参考。此外, 其他RAG框架如LightRAG也采用了相同的递归字符分块策略, 其代码实现可作为补充参考。

学术研究背景

“递归字符分割”是文本分割 (**Text Splitting**) 或句子分割 (**Sentence Splitting**) 这一经典NLP任务在工程上的具体应用。学术界的相关研究提供了更智能、基于语义的分割方法。

- **经典数据集**: 这些数据集多用于训练和评估智能分割模型, 相关论文提供了领域的基准:
 - **WebSplit**: 一个大规模合成数据集, 包含从知识图谱中提取的复杂句子及其拆分版本。
 - **WikiSplit**: 包含约100万个从维基百科编辑历史中自动生成的英语句子拆分对, 词汇量远大于WebSplit。
 - **BiSECT**: 包含英语、德语、法语和西班牙语四种语言, 共一百万个复杂句子及对应的简短句子。
 - **WikiSplit++**: 对WikiSplit的改进版本, 通过过滤不可靠的样本, 提高了数据的事实一致性和连贯性。
- **前沿方法综述**: 一篇名为“From Lengthy to Lucid: A Systematic Literature Review on NLP Techniques for Taming Long Sentences”的综述论文, 系统地回顾了处理长句子的方法, 将句子分割技术分为基于句法、基于篇章和基于语义的方法, 并提到了Transformer架构在该任务上的应用。

语义分块 (Semantic Splitting)

定义与基本原理

语义分块 (**Semantic Chunking**) 是一种根据文本的**语义连贯性**来动态确定分块边界的智能文本分割策略。它的核心目标不再是按固定长度切分, 而是将语义上紧密相关的句子聚合在一起, 形成一个完整、连贯的“话题单元”。

其基本原理可以概括为以下几个步骤:

1. **句子切分**: 首先将文档分割成单独的句子。
2. **向量化**: 使用预训练的嵌入模型 (如 `text-embedding-3-small` 或 `all-MiniLM-L6-v2`) 将每个句子转换为一个能代表其语义的向量 (即嵌入向量)。
3. **计算语义距离**: 依次计算相邻句子向量之间的相似度 (通常使用余弦相似度)。如果两个连续句子的意思相近, 相似度就高; 反之则低。
4. **检测断点**: 根据预设的阈值 (例如百分位数、标准差或一个固定值) 来检测语义的“突变”点。当相邻句子的相似度低于设定的阈值时, 算法判定此处发生了话题的显著转变, 并在此处进行分割。

简单的 Python 示例 (使用 LangChain)

以下代码展示了如何使用 LangChain 的 `SemanticChunker` 进行语义分块。这个示例使用的是 HuggingFace 的本地嵌入模型, 无需 OpenAI API Key 即可运行。

```
# 1. 导入必要的库
from langchain_experimental.text_splitter import SemanticChunker
from langchain_huggingface import HuggingFaceEmbeddings

# 2. 初始化嵌入模型（本地轻量级模型，用于计算句子语义）
# 'all-MiniLM-L6-v2' 是一个经典的、适合在CPU上运行的语义模型
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# 3. 创建语义分块器
# 设置断点检测类型为 "percentile"，即基于相似度分布的百分位数来确定断点
text_splitter = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="percentile"
    # 其他可选类型: "standard_deviation", "interquartile", "gradient"
)

# 4. 准备一段包含多个话题的长文本
long_text = """
人工智能是计算机科学的一个重要分支，旨在创造能够执行通常需要人类智能的任务的系统。
机器学习是实现人工智能的一种核心方法，它使计算机能够从数据中学习规律。
深度学习则是机器学习的一个子集，它使用多层神经网络来处理复杂的模式识别任务。
与此同时，篮球是一项广受欢迎的运动，NBA联赛汇集了全世界最顶尖的篮球运动员。
在篮球比赛中，球员们通过运球、传球和投篮来争夺分数，团队合作至关重要。
"""

# 5. 执行语义分块，返回语义连贯的文本块列表
chunks = text_splitter.split_text(long_text)

# 6. 查看结果
for i, chunk in enumerate(chunks):
    print(f"--- 块 {i+1} ---\n{chunk}\n")
```

代码输出分析：运行这段代码，你会看到文本被分成了两个清晰的块。第一个块包含了关于人工智能、机器学习和深度学习的内容，第二个块则是关于篮球运动的描述。这完美地体现了语义分块会根据话题的变化来切分文本。

可参考的论文与文献

语义分块是当前 RAG（检索增强生成）研究中的一个前沿方向，以下是一些相关的学术探讨：

- 核心概念来源：**语义分块的概念最初由 Greg Kamradt 在其关于“5 个级别嵌入分块”的视频教程中提出，后被 LlamaIndex 和 LangChain 等项目实现为实用工具。
- 改进算法：**一篇发表于《计算机技术与发展》的论文提出了一种融合大语言模型（LLM）的动态语义分块检索算法（HACR），旨在解决在处理复杂、主题交错的文档时语义边界模糊的问题。
- 全局优化方法：**《信息技术与信息化》期刊上的一篇论文则提出了一种基于动态规划的语义感知全局最优分块方法，利用全局信息来寻找最佳分块边界，以提升检索和生成任务的表现。
- 相关综述：**一篇题为“From Lengthy to Lucid: A Systematic Literature Review on NLP Techniques for Taming Long Sentences”的综述论文，系统地回顾了包括句子分割在内的各种长文本处理技术，可作为更广泛的背景阅读。

优缺点对比

维度	语义分块	传统分块 (如字符/递归分块)
核心逻辑	基于句子间的语义相似度动态分割	基于固定的文本长度或分隔符进行分割
优点	产生语义连贯的块，话题完整性好，利于精准检索和生成	简单、高效、成本极低，易于实现和控制
缺点	计算成本较高（需要调用嵌入模型）；阈值需要调试；性能优势在某些场景下不显著	容易切断语义，破坏话题完整性，可能产生语义不连贯的块
适用场景	对检索精度要求高的场景，如法律文本、科研论文、技术支持文档等	通用场景，对成本敏感或文本语义结构不复杂的场景

特别提示：有一篇名为《Is Semantic Chunking Worth the Computational Cost?》的论文通过实验指出，在部分检索和生成任务中，语义分块的高昂计算成本并未带来一致的性能提升，在某些场景下甚至与固定大小分块效果相当。因此，在实际项目中，建议根据具体需求和成本考量来权衡是否使用。

向量化与索引构建

文本向量化（Embedding）

文本向量化（Embedding）是构建RAG系统的核心步骤，也是让计算机真正“理解”文本语义的关键技术。

定义与基本原理

文本向量化（Text Embedding）是将文本（如单词、句子或文档）转换为**固定长度的数值数组（向量）**的过程。例如，一段文本可能被转换为一个768维的向

量： [0.023, 0.847, -0.129, ..., 0.352]。

其核心价值在于：通过大量文本的训练，语义相似的文本会被映射到向量空间中**距离相近**的位置。比如，“猫”和“狗”的向量会比较接近，而“汽车”则离它们很远。这样一来，机器就可以通过**计算向量之间的距离**（如余弦相似度）来判断文本的语义相关性。

📖 技术演进历程

文本向量化技术经历了从简单到智能的演进过程，了解这个脉络有助于你理解为什么现代RAG系统会选择特定的Embedding模型。

发展阶段	代表技术	核心思想	主要局限
早期统计方法	独热编码、词袋模型、TF-IDF	基于词频统计，将文本表示为高维稀疏向量。	高维稀疏， 无法捕捉语义关系 ，所有词被独立对待。
静态词向量时代	Word2Vec、GloVe	通过神经网络训练，将词映射到 低维稠密 的向量空间，语义相近的词向量也相近。	上下文无关 ：同一个词在不同语境下向量相同，无法处理“一词多义”。
动态词向量时代	BERT、Sentence-BERT	利用Transformer等深度模型， 根据上下文动态生成词向量 ，能捕捉一词多义。	计算资源消耗大，对长文本的处理存在挑战。

🔗 在LangChain中的简单应用

在LangChain中，使用Embedding模型非常简洁。以下是一个使用OpenAI模型的示例，但类似接口也适用于Cohere、HuggingFace等多种模型。

```
from langchain_openai import OpenAIEmbeddings

# 1. 初始化嵌入模型（需设置OPENAI_API_KEY环境变量）
embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")

# 2. 将单条查询文本向量化
query_text = "机器学习是什么？"
query_vector = embeddings.embed_query(query_text)

# 3. 将多个文档批量向量化（用于RAG的知识库）
doc_texts = [
    "人工智能是计算机科学的一个分支...",
    "机器学习是实现人工智能的一种核心方法..."
]
doc_vectors = embeddings.embed_documents(doc_texts)

# 输出向量维度，例如ada-002模型为1536维
print(f"查询向量维度: {len(query_vector)}")
print(f"文档向量数量: {len(doc_vectors)}")
```

📚 可参考的论文与文献

- **Word2Vec**: `Efficient Estimation of Word Representations in Vector Space` (Mikolov et al., 2013)。奠基之作，提出了CBOW和Skip-gram两种经典模型。
- **GloVe**: `GloVe: Global Vectors for Word Representation` (Pennington et al., 2014)。另一种广泛使用的词向量模型。
- **BERT**: `BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding` (Devlin et al., 2018)。开启了动态词向量时代，是理解现代Embedding技术绕不开的里程碑。
- **Sentence-BERT**: `Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks` (Reimers & Gurevych, 2019)。专门用于生成高质量句子向量的经典模型，广泛应用于RAG系统。

总结来说，文本向量化（Embedding）是连接人类语言和机器计算的桥梁。在RAG系统中，它使得**语义搜索**成为可能——即使两个句子没有相同的关键词，只要它们在“意思上”相似，就能被检索到。你的 `PyPDFLoader` 负责将PDF读成文本，而Embedding模型则负责将这些文本块变成可被检索的向量。

向量数据库与索引

以下是一段可直接运行的演示代码，它使用少量关于 RAG 的示例数据，演示了从文本向量化到索引构建的完整流程，并包含关键步骤的日志打印，方便你验证每一步是否成功。

```
# 1. 导入所需库
from langchain_community.embeddings import HuggingFaceEmbeddings # 用于文本向量化
from langchain_community.vectorstores import FAISS # FAISS 向量数据库
from langchain_core.documents import Document # LangChain 文档结构

# 2. 准备数据：自己编3-5条关于 RAG 的示例文档
docs = [
    Document(page_content="检索增强生成（RAG）是一种结合了信息检索与大型语言模型的技术。"),
    Document(page_content="RAG 的核心步骤包括：文档加载、文本切分、向量化、存储索引和检索生成。"),
    Document(page_content="向量数据库是 RAG 系统中用于高效存储和检索文档向量的关键组件。"),
    Document(page_content="常见的向量数据库有 FAISS、Chroma 和 Pinecone，其中 FAISS 适合本地快速原型验证。"),
    Document(page_content="文本向量化（Embedding）将文本转换为数值向量，便于计算语义相似度。")
]

# 3. 初始化嵌入模型（使用轻量级本地模型，无需 API Key）
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

print(f"✓ 已加载嵌入模型，模型名：sentence-transformers/all-MiniLM-L6-v2")
print(f"✓ 已准备 {len(docs)} 条示例文档。\\n")

# 4. 构建 FAISS 向量索引
# 将文档和嵌入模型传入，FAISS 会自动完成向量化并构建索引
vectorstore = FAISS.from_documents(docs, embeddings)

print(f"✓ FAISS 向量索引构建成功！")
print(f"✓ 索引中存储的向量总数：{vectorstore.index.ntotal}") # 打印索引中的向量总数，验证构建结果

# 5. （可选）简单检索测试，验证索引可用性
query = "什么是 RAG？"
retrieved_docs = vectorstore.similarity_search(query, k=2) # 检索最相关的 2 条文档

print(f"\\n--- 检索测试：查询 '{query}' ---")
print(f"检索到 {len(retrieved_docs)} 条相关文档：")
for i, doc in enumerate(retrieved_docs, 1):
    print(f"{i}. {doc.page_content}")
```

运行结果示例

当你运行这段代码时，你会看到类似以下的输出，说明索引已成功构建并可用于检索：

```
✓ 已加载嵌入模型，模型名：sentence-transformers/all-MiniLM-L6-v2
✓ 已准备 5 条示例文档。

✓ FAISS 向量索引构建成功！
✓ 索引中存储的向量总数：5

--- 检索测试：查询 '什么是 RAG？' ---
检索到 2 条相关文档：
1. 检索增强生成（RAG）是一种结合了信息检索与大型语言模型的技术。
2. RAG 的核心步骤包括：文档加载、文本切分、向量化、存储索引和检索生成。
```

代码中的关键点

- **本地嵌入模型**：使用 `sentence-transformers/all-MiniLM-L6-v2` 可以完全在本地运行，无需设置 API Key，非常适合快速原型开发。
- **FAISS 自动构建**：`FAISS.from_documents` 方法一步完成了向量化、索引构建和存储，封装了底层细节，让你可以专注于业务逻辑。
- **验证步骤**：代码中通过 `vectorstore.index.ntotal` 和一次检索测试，直观地验证了索引的构建结果和可用性。

在RAG系统中，向量数据库是存储和检索向量化文档的核心基础设施。如果说文本向量化是把文档变成了一串数字，那么向量数据库和索引就是为这海量数字打造的一套高效检索系统。

💡 核心定义与价值

向量数据库是一种专门用于存储、索引和检索高维向量数据的数据库系统。它的核心价值在于能够对海量向量进行**高效的相似性搜索**，使你可以在百万甚至十亿级的数据中快速找到与查询最相似的文档。

而**向量索引**是实现这一高效搜索的关键技术。它本质上是一种特殊的数据结构，用于组织向量数据，将原本需要遍历所有数据的"暴力搜索"，转变为一种"智能导航"，从而极大提升查询速度。

⚖️ 索引的核心权衡：精度 vs. 速度

向量索引的设计都基于一个核心权衡：**用可接受的精度损失，换取数量级的查询速度提升**。绝大多数向量索引采用的近似最近邻搜索（ANNS, Approximate Nearest

Neighbor Search) 算法，正是为了达成这个权衡。

这里有几个关键概念需要理解： * **精确最近邻搜索 (NN, Nearest Neighbor Search)**：就像遍历整本字典，保证找到完全正确的结果，但数据量大时速度极慢，在百万级数据上每次查询可能耗时数秒。 * **近似最近邻搜索 (ANNS, Approximate Nearest Neighbor Search)**：则像是先查目录再翻书页，通过智能的数据结构快速锁定一个很小的范围进行搜索。其准确性通常用**召回率 (Recall)** 衡量，指返回结果中有多少是真正的最近邻。对于多数AI应用，95-99%的召回率是可接受的，因为速度可以提升百倍甚至千倍。

主流向量索引技术解析

了解了几种主流索引的工作原理和适用场景，能帮你更好地做出选择。

1. FLAT：精确的"笨办法"

FLAT 是最基础的索引，它对所有向量进行**暴力搜索 (brute-force search)**，不做任何近似处理，因此结果**100%精确**。

它就像一个拥挤的房间，你要找到最像你的人，就得问遍房间里每一个人。当数据量超过一万行时，它的效率会急剧下降，查询变得非常缓慢。

- 适用场景**：数据量极小（如 < 1万条）且对精度有绝对要求的场景。

2. IVF (Inverted File)：高效的"分而治之"

IVF（倒排文件索引）的核心思想是**"分而治之"**。它通过K-means等聚类算法，将所有向量划分成许多个**"桶"**（或称聚类）。

查询时，它先找出和你的查询向量最接近的几个桶，然后只在这些桶内部进行精确搜索，从而极大地减少了搜索范围。

- 如何理解**：就像在一个巨大的图书馆里，你要找一本关于"人工智能"的书。你不会从第一本书开始翻，而是先去"计算机科学"这个分类区域找，大大节省了时间。
- 重要参数**：`nprobe` 控制了搜索时考虑的桶数量。增大它，搜索范围变大，精度更高但速度变慢；减小则反之。
- 适用场景**：非常适合百万级以上的大规模、高维数据集，是许多生产系统的首选。

3. IVF的变种：引入量化，节省存储

为了进一步优化，IVF常常与量化技术结合，用更少的内存存储向量。 * **IVF_FLAT**：基础版，不对向量进行压缩，精度最高，但存储和内存占用也最大。 * **IVF_SQ8**：通过**标量量化 (Scalar Quantization)** 将每个向量的维度从4字节浮点数压缩为1字节整数，内存占用减少约70-75%。 * **IVF_PQ**：使用**乘积量化 (Product Quantization)**，将高维向量切分成子向量分别聚类编码，可实现4-32倍的压缩比，**内存效率最高**，适合超大规模或内存受限的场景，但精度损失比IVF_SQ8稍大。

4. HNSW：快速智能的"导航图"

HNSW（分层可导航小世界图）是当前公认**查询速度最快**的索引之一。它构建了一个多层图结构：顶层节点稀疏，像一张高速公路网，用于快速定位大致区域；底层节点密集，像城市里的小巷，用于精确定位。

搜索时，查询向量从顶层开始，像用导航一样，贪婪地沿着最相似的边向底层移动，最终快速锁定最近邻。

- 如何理解**：这就像你要去一个陌生的城市找一家餐厅。你不会一条街一条街地找，而是先用导航在"高速公路层"定位到那个城市区域，然后在"城市道路层"找到具体街区，最后在"街区层"找到餐厅门口。
- 关键参数**：`M`（每个节点的最大连接数，越大精度越高但内存消耗越大）和 `efConstruction/ef`（构建/搜索时考虑的候选节点数，影响图质量与速度）。
- 适用场景**：对查询延迟要求极高、数据集规模中等（如百万级）、内存充裕的场景。

如何选择：核心考量因素

选择合适的索引需要综合权衡：

考量维度	关键点	建议方向
数据规模	数据量多大？	< 1万：FLAT；< 100万：HNSW或IVF_FLAT；> 100万：IVFPQ或IVF_SQ8。
性能要求	更看重速度还是精度？	速度优先：HNSW；精度优先但需控制成本：IVF_FLAT；需要极致的存储和速度平衡：IVFPQ。
硬件资源	内存是否充裕？	内存充足：HNSW；内存受限：IVFSQ8或IVFPQ。
向量维度	向量是低维 (<128) 还是高维 (>512) ？	低维：HNSW表现可能更优；高维（存在维度灾难）：IVF或PQ系索引更适配。

总结

向量索引是向量数据库的性能核心，选择哪种索引本质上是在**查询速度**、**内存/存储开销**和**召回精度**之间做权衡。对于大多数刚起步的RAG项目，可以先从基于IVF的索引（如IVF_FLAT）开始，它在性能、精度和成本之间提供了很好的平衡；而如果对查询速度有极致追求且资源充足，HNSW会是更优的选择。

检索器设计与优化

在RAG系统中，检索器（Retriever）是连接向量数据库与大语言模型的桥梁，负责根据用户的问题，从知识库中找出最相关的文档片段。

检索器的核心职责是接收一个查询（Query），并返回一组相关的文档（Documents）。在LangChain中，它被抽象为一个统一的接口，屏蔽了底层向量数据库的具体

操作，让你可以像调用一个函数一样进行检索

基础检索

基础检索是 RAG 系统的核心环节，其过程可以理解为将用户的问题转化为向量，然后在向量数据库中寻找最相似的文档片段。下面我用一个完整的例子来展示这一过程。

基础检索的四个步骤

基础检索的过程可以分解为以下四个清晰的步骤：

1. **查询向量化**：使用与索引文档时相同的嵌入模型，将用户输入的文本问题转换为向量。
2. **相似度计算**：在向量数据库中，计算查询向量与所有已索引文档向量之间的相似度（通常使用余弦相似度）。
3. **排序与筛选**：根据相似度得分对所有文档进行降序排序，并选出得分最高的前 `k` 个文档（`k` 是一个预设参数）。
4. **返回结果**：将这 `k` 个最相关的文档（通常包含原始文本和元数据）作为检索结果返回。

完整代码示例

下面的代码展示了从创建向量库到执行检索的完整流程，并包含了详细的注释。

```
# 1. 导入所需库
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document

# 2. 准备示例文档（关于 RAG 的知识）
docs = [
    Document(page_content="检索增强生成（RAG）结合了信息检索与大语言模型，能有效减少模型幻觉。"),
    Document(page_content="RAG 系统的核心流程包括：文档加载、文本切分、向量化、索引构建和检索生成。"),
    Document(page_content="向量数据库是 RAG 中存储文档向量的关键组件，常见的有 FAISS、Chroma 和 Pinecone。"),
    Document(page_content="文本向量化（Embedding）将文本转换为数值向量，用于计算语义相似度。"),
    Document(page_content="检索器（Retriever）是连接向量库与大语言模型的桥梁，负责查询并返回相关文档。")
]

# 3. 初始化嵌入模型（本地轻量级模型）
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

# 4. 构建 FAISS 向量索引
vectorstore = FAISS.from_documents(docs, embeddings)

# 5. 创建检索器（配置为返回最相似的 2 个文档）
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

# 6. 执行基础检索
query = "什么是 RAG?" # 用户的问题
retrieved_docs = retriever.invoke(query) # 调用检索器

# 7. 打印检索结果，展示基础检索过程
print(f"查询语句: {query}\n")
print(f"检索到 {len(retrieved_docs)} 个相关文档:\n")
for i, doc in enumerate(retrieved_docs, 1):
    # 打印文档内容和相似度得分（FAISS 默认返回带分数的结果）
    print(f"--- 结果 {i} ---")
    print(f"内容: {doc.page_content}")
    print(f"相似度得分: {doc.metadata.get('score', 'N/A')}\n")
```

运行结果示例

运行以上代码，你会看到类似下面的输出（具体得分可能略有不同）：

```
查询语句：什么是 RAG？

检索到 2 个相关文档：

--- 结果 1 ---
内容：检索增强生成（RAG）结合了信息检索与大语言模型，能有效减少模型幻觉。
相似度得分：0.89

--- 结果 2 ---
内容：RAG 系统的核心流程包括：文档加载、文本切分、向量化、索引构建和检索生成。
相似度得分：0.76
```

代码的关键点说明

- **向量化与索引构建**: `FAISS.from_documents` 这一步自动完成了文档的向量化转换和 FAISS 索引的构建。
- **检索器创建**: `vectorstore.as_retriever(search_kwargs={"k": 2})` 创建了一个检索器, 并通过 `k=2` 指定了返回文档的数量。
- **相似度得分**: `doc.metadata['score']` 显示了每个文档与查询的相似度, 这个分数通常在 0 到 1 之间, 越接近 1 表示越相关。
- **检索过程透明化**: 如果你想查看更详细的检索日志 (如每个文档的具体得分和排名), 可以启用 LangChain 的调试模式:

```
python import langchain langchain.debug = True
```

这个例子完整地展示了基础检索的流程。在实际项目中, 你可以通过调整 `k` 值、更换嵌入模型或结合更高级的检索策略来优化检索效果。

进阶检索

进阶检索技术是RAG系统从"能用"走向"好用"的关键。当基础向量检索在精确匹配、结果多样性等方面遇到瓶颈时, **混合检索**和**重排序**这两项核心技术能有效提升召回质量。

混合检索 (Hybrid Search) : 精耕细作

💡 为什么需要混合检索?

纯向量检索擅长理解语义, 能找到意思相近但用词不同的内容。但它有个明显的盲区: 对于**专有名词**、**产品型号**、**缩写词**等需要精确匹配的信息, 向量检索可能找不到或找不准。

举个例子, 用户搜索"MacBook Pro M3", 向量检索可能召回一堆关于"笔记本电脑"或"苹果产品"的泛泛内容, 而关键词检索能精准命中包含这个确切型号的文档。

混合检索就是为解决这个问题而生: 将**向量检索 (语义)**和**关键词检索 (精确)**结合起来, 取长补短。

🧠 工作原理与融合方法

混合检索在索引阶段同时建立两套索引: 一套是向量索引 (FAISS), 另一套是关键词索引 (如BM25的倒排索引)。查询时, 两路检索并行执行, 各返回一份排序结果, 然后通过融合算法合并成一份最终排名。

最常用的融合算法是**倒数排名融合 (RRF, Reciprocal Rank Fusion)**。

- **核心思想**: 不看原始得分 (因为语义和关键词的打分体系不同), 只看文档在两份排名中的**位置**。
- **计算公式**: $RRF_score(d) = \sum \frac{1}{(k + rank(d))}$, 其中 `k` 是一个平滑常数 (通常设为60)。
- **优点**: 简单、鲁棒, 无需调参, 能有效融合不同检索系统的优势。

🐍 代码示例: 简单混合检索 (RRF融合)

```
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import OpenAIEmbeddings
from rank_bm25 import BM25Okapi
from langchain_core.documents import Document

# 1. 准备文档
docs = [
    Document(page_content="MacBook Pro M3 拥有最新的M3芯片。"),
    Document(page_content="苹果公司发布了新款笔记本电脑。"),
]

# 2. 构建向量索引 (FAISS)
vectorstore = FAISS.from_documents(docs, OpenAIEmbeddings())
# 3. 构建BM25关键词索引
tokenized_corpus = [doc.page_content.split() for doc in docs]
bm25 = BM25Okapi(tokenized_corpus)

def hybrid_search(query: str, k: int = 2):
    # 4. 执行两路检索
    vector_results = vectorstore.similarity_search_with_score(query, k=10) # 召回更多候选
    vector_rank = {doc.page_content: idx for idx, (doc, _) in enumerate(vector_results)}

    tokenized_query = query.split()
    bm25_scores = bm25.get_scores(tokenized_query)
    bm25_rank = {docs[i].page_content: idx for idx, score in enumerate(sorted(range(len(bm25_scores)), key=lambda i: bm25_scores[i]))}

    # 5. RRF融合
    rrf_scores = {}
    for doc in docs:
        rrf_score = 0
        if doc.page_content in vector_rank:
            rrf_score += 1 / (60 + vector_rank[doc.page_content])
        if doc.page_content in bm25_rank:
            rrf_score += 1 / (60 + bm25_rank[doc.page_content])
        rrf_scores[doc.page_content] = rrf_score

    # 6. 排序并返回Top-K
    sorted_results = sorted(rrf_scores.items(), key=lambda item: item[1], reverse=True)
    return sorted_results[:k]

print(hybrid_search("M3芯片 MacBook"))
# 预期输出: [('MacBook Pro M3 拥有最新的M3芯片.', RRF分数), ...]
```

重排序（Rerank）：优中选优

💡 为什么需要重排序？

混合检索虽然召回了更多候选，但可能数量较多（比如Top-50），且排序可能不够精准。这时需要**重排序**：用一个更强大但计算成本更高的模型，对初选结果**重新打分和排序**，把最相关的文档推到最前面。

🧠 工作原理

重排序在RAG流程中的位置很清晰：

- 1. **初步检索（Recall）**：使用混合检索或向量检索快速召回一个较大的候选集（如Top-50）。
- 2. **重排序（Rerank）**：将用户问题和每个候选文档组合成 `(Query, Document)` 对，输入给一个交叉编码器（**Cross-Encoder**）模型，得到精准的相关性分数。
- 3. **筛选（Filter）**：根据重排序分数，选出最终的Top-N（如Top-3）传递给大模型。

⚖️ 重排序模型的选择

模型	类型	特点	适用场景
BGE-Rerank (base/large)	开源	中文表现优秀，可本地部署，精度高但推理较慢	对数据隐私有要求、可接受一定延迟的场景
Cohere Rerank	商业API	多语言支持好，集成方便，性能强大	追求快速集成、多语言环境的商业项目
LLM-as-Reranker	自建/API	使用GPT等大模型对Top-5文档进行最终排序，精度最高，但成本昂贵	对准确性有极致要求、预算充足的场景

💎 总结：进阶检索流程

将这两项技术结合，就形成了一个**多阶段检索流水线**：

1. 用户查询输入。
2. 混合检索：同时执行向量检索（语义）和BM25关键词检索（精确）。
3. 结果融合：使用RRF算法合并两路结果，得到初步排序的候选文档列表（如Top-50）。
4. 重排序：使用交叉编码器模型（如BGE-Rerank）对候选文档重新打分，选出最相关的Top-N（如Top-3）。
5. 生成答案：将精排后的文档作为上下文，交给大模型生成最终回答。

通过这套组合拳，RAG系统的检索精度和鲁棒性可以得到显著提升，更好地应对生产环境中的复杂查询。

高阶检索简单介绍

在混合检索和重排序之外，RAG领域确实涌现了更多前沿的高阶检索策略。这些策略不再满足于“快速找到相似的文本块”，而是试图让系统变得更智能、更全面，像人类专家一样去理解和查找信息。

🔍 查询转换 (Query Transformation)

这是一种在检索开始前就介入的策略，通过优化用户的问题本身，让检索的起点更精准。

- **HyDE (Hypothetical Document Embeddings)**：它的思路很巧妙，不是直接用用户的问题去检索，而是先让大语言模型（LLM）根据问题生成一个“假设性”的答案文档，然后用这个假设答案的向量去检索。因为假设答案包含了更丰富的上下文和关键词，往往比原始问题能更好地命中相关文档。
- **多查询 (Multi-Query)**：在处理复杂问题时，LLM会被用来将用户的一个原始问题分解成多个不同角度的子问题，然后对每个子问题分别进行检索，最后汇总所有结果。这能有效避免遗漏关键信息。

📊 知识图谱检索 (Graph RAG)

这是当前最受关注的方向之一，旨在解决传统向量检索在需要多步推理的复杂问题上的不足。它被一些学术综述视为与向量检索并列的核心RAG范式。

- **核心思想**：Graph RAG不再仅仅依赖文本的向量相似度，而是从文档中抽取实体（如人名、地名、概念）及其之间的关系，构建一个知识图谱。
- **工作方式**：查询时，系统会沿着图谱中的关系进行**多跳推理**。例如，回答“A公司的CEO毕业于哪所大学？”时，需要先找到“A公司”的“CEO”是谁，再找到该CEO的“毕业院校”。这种方法能提供更精确、可解释的信息，尤其适合处理跨文档、需要逻辑关联的复杂查询。

🤖 代理式检索 (Agentic RAG)

这是将大语言模型的推理和决策能力与检索过程深度结合的产物，代表了一种从“被动检索”到“主动探索”的范式转变。

- **核心思想**：不再将检索视为一次性的动作，而是交给一个AI“代理”（Agent）来全权负责。这个Agent被赋予一组工具，如搜索、打开文档、跳转章节、精确匹配关键词等。
- **工作方式**：Agent会先进行初步搜索，然后**自主判断**信息是否充足。如果不够，它会像一个研究员一样，主动决定“打开”哪份文档，跳转到相关章节“阅读”，甚至跨文档交叉比对，在**多步循环**中不断收集证据，直到它认为自己掌握了足够的信息后，才生成最终答案。
- **效果**：这种模式在处理长文档、跨章节、复杂表格等场景中表现出色。例如，Mistral AI的Agentic Search在金融文档问答的测试中，准确率相比传统方案提升了约3倍，最高达到86%，同时Token消耗也大幅减少。

🌐 多模态检索 (Multimodal RAG)

现实世界的信息不止于文本，还包含图像、图表、视频等。多模态RAG的目标就是打破传统RAG“模态单一”的限制。

- **核心技术**：最新的研究方向如 **UniversalRAG**，提出了一种“智能路由”框架。它不再试图将所有不同模态的数据强行塞入一个向量空间，而是为**文本、图像、视频**等不同模态的知识分别建立独立的索引库。
- **工作方式**：系统接收到查询后，会先由一个“路由器”判断回答这个问题最需要哪种模态的知识，然后将查询引导至对应的知识库进行检索，从而巧妙地避免了不同模态之间的“语义鸿沟”问题。

📊 策略对比与总结

高阶检索策略	核心思路	主要优势	适用场景
查询转换	在检索前优化用户问题（如生成假设答案、分解子问题）	提升检索起点质量，弥补用户表述不清。	用户问题简短、模糊或复杂多面时。
知识图谱检索	利用实体关系图谱进行多跳推理，寻找精准答案	答案更精确、可解释，擅长处理复杂逻辑推理。	需要跨文档关联、多步推理的问答（如金融、法律分析）。
代理式检索	赋予AI代理多步规划、决策和使用工具的能力，主动探索信息	突破一次性检索限制，能像人一样深入研究，召回率极高。	处理海量、结构复杂的长文档（如年报、技术手册）。
多模态检索	为不同模态（文本/图像/视频）建立独立索引，并由路由器智能分发	解锁对非文本信息的检索和理解能力。	查询同时涉及多种信息形式（如“描述这张图表的内容”）。

这些前沿策略各有侧重，也并非互斥。在实际工程中，一个成熟的生产级RAG系统往往会组合使用其中多种技术。例如，先用**查询转换**优化问题，再通过**混合检索**快速召回，然后利用**重排序**进行精筛，最后对于最复杂的请求，可能还会触发**代理式检索**进行深度探索。

简单的生成器

在RAG系统中，**生成器（Generator）** 是最终负责产出答案的组件。它接收检索器找到的相关文档和用户问题，由大语言模型（LLM）生成一个连贯、准确的回答。

下面是一个最简的生成器实现示例，你可以直接复制运行，快速验证整个RAG流程。

```
# 1. 导入所需库
from langchain_openai import ChatOpenAI # OpenAI 聊天模型
from langchain_core.prompts import ChatPromptTemplate # 提示词模板
from langchain_core.output_parsers import StrOutputParser # 输出解析器
from langchain_community.vectorstores import FAISS # 向量数据库
from langchain_community.embeddings import OpenAIEmbeddings # 嵌入模型
from langchain_core.documents import Document # 文档结构
import os

# 2. 准备数据：编几条关于 RAG 的示例文档
docs = [
    Document(page_content="检索增强生成（RAG）是一种结合信息检索与大语言模型的技术，能有效减少模型幻觉。"),
    Document(page_content="RAG 的核心步骤包括：文档加载、文本切分、向量化、索引构建、检索和生成。"),
    Document(page_content="生成器通常使用大语言模型（如 GPT、Claude 或开源模型），根据检索到的上下文生成最终答案。")
]

# 3. 构建向量索引和检索器（模拟已有的检索步骤）
embeddings = OpenAIEmbeddings() # 初始化 OpenAI 嵌入模型
vectorstore = FAISS.from_documents(docs, embeddings) # 构建 FAISS 向量索引
retriever = vectorstore.as_retriever(search_kwargs={"k": 2}) # 创建检索器，返回最相似的2个文档

# 4. 检索：根据用户问题获取相关文档
question = "什么是 RAG？它的核心步骤有哪些？" # 用户问题
retrieved_docs = retriever.invoke(question) # 执行检索，获得相关文档

print(f"--- 检索到的 {len(retrieved_docs)} 个相关文档 ---")
for i, doc in enumerate(retrieved_docs, 1):
    print(f"{i}. {doc.page_content}\n")

# 5. 构建生成器：使用大语言模型生成最终答案
# 定义提示词模板，将检索到的文档和用户问题组合起来
template = """
你是一个知识渊博的助手。请基于以下提供的上下文信息，回答用户的问题。
如果上下文信息不足以回答问题，请坦诚地告诉用户。

上下文信息：
{context}

用户问题：
{question}

请给出清晰、准确的回答：
"""
prompt = ChatPromptTemplate.from_template(template) # 创建提示词模板

# 初始化 LLM（这里使用 OpenAI 的 gpt-3.5-turbo，需要设置 OPENAI_API_KEY 环境变量）
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# 将提示词模板、LLM 和输出解析器串联成一个处理链
chain = prompt | llm | StrOutputParser()

# 6. 生成回答：将检索到的文档和问题传入处理链
# 将 Document 对象列表合并为一个字符串作为上下文
context = "\n".join([doc.page_content for doc in retrieved_docs])
response = chain.invoke({"context": context, "question": question})

print("\n--- 生成的最终答案 ---")
print(response)
```

代码运行结果示例

运行这段代码，你将看到类似以下的输出：

--- 检索到的 2 个相关文档 ---

- 检索增强生成（RAG）是一种结合信息检索与大语言模型的技术，能有效减少模型幻觉。
- RAG 的核心步骤包括：文档加载、文本切分、向量化、索引构建、检索和生成。

--- 生成的最终答案 ---

RAG（检索增强生成）是一种将信息检索与大语言模型相结合的技术，旨在通过检索外部知识来增强模型生成内容的质量，减少幻觉现象。其核心步骤包括文档加载、文本切分、向量化、索引构建、检索和生成。

代码关键点解析

- 组合模式：**`chain = prompt | llm | StrOutputParser()` 是LangChain的核心设计，使用管道运算符 `|` 将提示词模板、语言模型和输出解析器串联成一个易于使用和调试的链。
- 提示词工程：**模板中的 `{context}` 和 `{question}` 是占位符，在调用 `chain.invoke()` 时会被真实数据替换。这里明确指示模型在知识不足时需承认，可以增强回答的可靠性。
- 简单可控：**此示例为你后续优化RAG流程提供了一个清晰、可控的起点。你可以轻松地替换不同的LLM（如使用 `ChatAnthropic` 或 `HuggingFaceHub`），调整提示词模板，或修改检索的 `k` 值来优化结果。

希望这个示例能帮你快速搭建起RAG系统的原型。

提示词模板

💡 定义与核心价值

提示词模板（Prompt Template） 是用于构建发送给大语言模型（LLM）的输入的结构化方法。它的核心价值在于将固定的指令框架与动态的用户输入、检索到的上下文分离开来，使你能够像使用函数一样，通过传入不同的变量来生成格式统一、内容精准的提示词。

在RAG系统中，提示词模板扮演着“指挥官”的角色，它告诉模型：你的任务是什么（角色定位）、可以参考哪些资料（上下文）、需要解决什么问题（用户输入）、以及应该以什么格式输出（格式约束）。

🧩 核心组成部分

一个标准的RAG提示词模板通常包含以下关键部分：

组成部分	作用	示例
系统/角色设定	定义模型的角色、任务目标和行为准则	“你是一位专业的金融分析师...”
上下文占位符	注入检索到的相关文档片段，为模型提供事实依据	“参考以下信息：\n{context}”
用户问题占位符	接收用户的具体提问	“用户的问题是：{question}”
输出格式指令	规定回答的结构、格式或风格	“请用简洁的要点列出答案。”

🐍 在 LangChain 中的三种创建方式

LangChain 提供了灵活的方式来创建提示词模板，以下是最常用的三种。

1. 基础模板： `PromptTemplate`

适用于简单的单变量场景，是最直接的方式。

```
from langchain_core.prompts import PromptTemplate

# 创建一个包含单个变量 'topic' 的模板
template = PromptTemplate.from_template("请用中文简要解释什么是 {topic}。")

# 通过传入变量值来格式化提示词
formatted_prompt = template.format(topic="向量数据库")
print(formatted_prompt)
# 输出：请用中文简要解释什么是 向量数据库。
```

2. 聊天模板： `ChatPromptTemplate`

专为聊天模型设计，支持定义不同角色（如系统、用户、助手）的消息，更符合现代LLM的交互范式。

```
from langchain_core.prompts import ChatPromptTemplate

# 创建一个包含系统消息和用户消息的聊天模板
template = ChatPromptTemplate.from_messages([
    ("system", "你是一位专业的知识库助手。请根据以下提供的上下文信息回答用户的问题。如果上下文不足，请明确告知。"),
    ("user", "上下文信息: \n{context}\n\n用户问题: \n{question}")
])

# 格式化模板（注意：聊天模板使用 format_messages 或 invoke）
# 实际使用时，通常直接传入链（chain）中，由 LangChain 自动处理
```

3. 少样本模板： `FewShotPromptTemplate`

当你需要给模型提供几个示例（Few-shot Examples）来引导它的输出风格或推理方式时使用。

```
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate

# 1. 定义示例
examples = [
    {"question": "什么是RAG?", "answer": "RAG是检索增强生成..."},
    {"question": "向量数据库的作用?", "answer": "向量数据库用于存储..."}
]

# 2. 定义每个示例的格式化模板
example_template = PromptTemplate.from_template("问题: {question}\n答案: {answer}")
example_prompt = PromptTemplate(input_variables=["question", "answer"], template="问题: {question}\n答案: {answer}")

# 3. 创建少样本模板
few_shot_template = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix="请根据以下示例的格式回答用户问题。",
    suffix="问题: {input}\n答案: ",
    input_variables=["input"]
)

print(few_shot_template.format(input="嵌入模型是什么?"))
```

🌟 高阶技巧简单介绍

1. 使用 `partial` 预置固定变量

对于需要在多个地方使用、但内容固定的变量（如固定的日期、系统规则），可以提前“预填”好，这样在调用时就可以不再重复传入。

```
from langchain_core.prompts import PromptTemplate
from datetime import datetime

# 创建一个模板，其中 current_date 变量被预置为当前日期
template = PromptTemplate.from_template("今天是 {current_date}。请告诉我今天适合做什么?")
partial_template = template.partial(current_date=datetime.now().strftime("%Y-%m-%d"))

# 调用时只需传入另一个变量（如果有的话）
formatted = partial_template.format()
print(formatted)
# 输出: 今天是 2026-08-24。请告诉我今天适合做什么?
```

2. 在提示词中加入“思维链”引导

通过在模板中加入“让我们一步一步思考”（Let's think step by step）等引导语，可以促使模型在回答前进行推理，尤其适合逻辑复杂的问题。

```
reasoning_template = ChatPromptTemplate.from_messages([
    ("system", "你是一位逻辑推理专家。"),
    ("user", "问题: {question}\n\n请一步一步地思考，并给出最终答案。")
])
```

💡 总结与最佳实践

一个好的提示词模板能让RAG系统的效果事半功倍。在设计时，请记住以下几点：

- 明确角色定位：**清晰告诉模型“你是谁”，如“你是一位法律顾问”或“你是一位技术支持工程师”。
- 提供足够的上下文：**确保 `{context}` 占位符能接收到充足、相关的检索结果。
- 设定清晰的边界：**当知识库中没有相关信息时，指导模型坦诚承认“我不知道”，可以有效避免模型“幻觉”。
- 格式要具体：**如果需要模型以JSON、Markdown或要点列表输出，务必在模板中明确指定。

LangChain 的提示词模板体系为你提供了强大的工具，使提示词工程从“复制粘贴”的作坊模式，升级到“可重用、可组合、可版本控制”的工程模式，是构建专业RAG应用的必备基础。

多轮对话

💡 定义与核心价值

多轮对话（Multi-turn Dialogue） 在RAG系统中，是指系统能够记住并理解用户在多次交互中提出的问题，并基于完整的对话历史来生成连贯、准确的回答。它的核心价值在于让AI从“一次性问答工具”升级为能够进行连贯、有上下文意识的对话伙伴。

与单轮问答相比，多轮对话的关键突破在于它赋予了系统“记忆”能力。这使得用户可以进行追问（例如，“那它的价格呢？”）、指代消解（例如，“它支持哪些功能？”中的“它”指代前文提到的产品），从而获得更自然、更高效的交互体验。

🔧 实现多轮对话的核心机制

在LangChain中，实现多轮对话主要依赖以下两个核心概念：

1. 对话窗口（Chat History / Message History）

这是存储所有历史消息（包括用户的问题和AI的回复）的容器。在每次新的对话中，系统会将**整个或部分历史消息**与当前问题一起打包，发送给LLM。

2. 内存（Memory）

内存是一个更高级的概念，它不仅负责存储历史消息，还负责**管理、压缩和总结**这些历史信息，以防止对话过长时超出LLM的上下文窗口限制（Token限制）。

工作原理示意图：

用户：“RAG是什么？” 助手：“RAG是检索增强生成...” ↓ （系统将整个对话存入内存） 用户：“它的核心步骤有哪些？”（新问题） ↓ （系统从内存中取出历史对话） ↓ （构造新的提示词）

📁 完整的代码示例

下面的示例展示了一个支持多轮对话的RAG系统核心流程。它使用LangChain的 `ChatMessageHistory` 和 `RunnableWithMessageHistory` 来管理对话记忆。

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_core.documents import Document

# 1. 准备数据和检索器（与之前相同）
docs = [
    Document(page_content="RAG（检索增强生成）结合了信息检索与大语言模型。"),
    Document(page_content="RAG的核心步骤包括：文档加载、文本切分、向量化、索引构建、检索和生成。"),
    Document(page_content="向量数据库是RAG的关键组件，如FAISS、Chroma等。")
]
vectorstore = FAISS.from_documents(docs, OpenAIEmbeddings())
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

# 2. 定义提示词模板，关键：使用 MessagesPlaceholder 来注入历史信息
template = """
你是一位知识渊博的助手。请根据以下提供的上下文信息回答用户的问题。
如果上下文不足，请坦诚告知。

上下文信息：
{context}

历史对话（仅供你理解上下文，无需重复）：
{history}

用户问题：
{question}

请给出清晰、准确的回答：
"""
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一位知识渊博的助手。请根据上下文回答问题。"),
    MessagesPlaceholder(variable_name="history"), # 历史消息将插入到这里
    ("human", "上下文信息：\n{context}\n\n用户问题：\n{question}")
])

# 3. 构建基础处理链（检索 + 生成）
def format_docs(docs):
```

```

        return "\n".join([doc.page_content for doc in docs])

# 创建一个简单的链：检索上下文 -> 格式化 -> 调用LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# 注意：这里我们使用一个更简洁的链，将检索和生成分开，便于观察
def retrieve_and_respond(inputs):
    question = inputs["question"]
    history = inputs.get("history", [])

    # 检索相关文档
    retrieved_docs = retriever.invoke(question)
    context = "\n".join([doc.page_content for doc in retrieved_docs])

    # 格式化提示词
    formatted_prompt = prompt.format_messages(
        history=history,
        context=context,
        question=question
    )

    # 调用LLM生成回答
    response = llm.invoke(formatted_prompt)
    return response.content

# 4. 包装为带记忆的对话链
# 创建内存存储，用于保存每个会话的历史
store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

# 使用 RunnableWithMessageHistory 包装我们的处理函数
# 注意：这里我们需要将 retrieve_and_respond 适配为Runnable
from langchain_core.runnables import RunnableLambda

# 将函数包装为Runnable
runnable = RunnableLambda(retrieve_and_respond)

# 添加消息历史管理
with_history = RunnableWithMessageHistory(
    runnable,
    get_session_history,
    input_messages_key="question",      # 输入中的问题键
    history_messages_key="history"      # 历史消息的键
)

# 5. 模拟多轮对话
session_id = "user_123" # 用用户ID作为会话标识

# 第一轮对话
print("用户：RAG是什么? ")
response1 = with_history.invoke(
    {"question": "RAG是什么? "},
    config={"configurable": {"session_id": session_id}}
)
print(f"助手：{response1}\n")

# 第二轮对话（追问）
print("用户：它的核心步骤有哪些? ")
response2 = with_history.invoke(
    {"question": "它的核心步骤有哪些? "},
    config={"configurable": {"session_id": session_id}}
)
print(f"助手：{response2}\n")

# 第三轮对话（指代）
print("用户：那向量数据库在其中的作用是什么? ")
response3 = with_history.invoke(
    {"question": "那向量数据库在其中的作用是什么? "},
    config={"configurable": {"session_id": session_id}}
)
print(f"助手：{response3}\n")

```

运行以上代码，你将看到类似下面的对话过程。注意助手能够理解每一轮对话中的指代（如“它”、“那”），这得益于历史消息的传递。

用户：RAG是什么？

助手：RAG是检索增强生成的缩写，它是一种结合信息检索与大语言模型的技术。

用户：它的核心步骤有哪些？

助手：RAG的核心步骤包括：文档加载、文本切分、向量化、索引构建、检索和生成。

用户：那向量数据库在其中的作用是什么？

助手：向量数据库是RAG的关键组件，用于存储和检索文档的向量表示。它使得系统能够快速找到与用户问题语义最相关的文档片段。

💡 关键点总结与最佳实践

概念	说明
会话ID	用于区分不同用户或会话，是实现多用户并发的关键。
消息历史存储	可选择内存（ <code>InMemoryChatMessageHistory</code> ）或持久化存储（如Redis、数据库）。
提示词设计	必须使用 <code>MessagesPlaceholder</code> 占位符来动态注入历史消息，这是LangChain的标准做法。
上下文窗口管理	当对话轮次过多时，需要考虑使用 <code>ConversationSummaryMemory</code> 等工具对历史进行总结压缩，避免超出Token限制。
RAG中的记忆	在RAG中，通常只记忆对话历史，而不记忆每次检索到的文档。因为每次检索都会根据最新问题重新获取上下文。

通过这种方式，你就构建了一个既能访问外部知识库，又能流畅进行多轮对话的RAG助手。这是搭建聊天机器人、智能客服等应用的核心基础。如果你需要实现更复杂的记忆策略（如总结历史、用户偏好记忆等），可以在现有框架上进一步扩展。

更多扩展

基于你提出的五个RAG分类，我结合文档内容为你进行了补充定义和特点梳理，并建立了一个对比表格。

📚 RAG 系统演进五阶段详解

1 基础 RAG（Naive RAG）

定义：采用标准的线性流程 "查询 → 检索 → 生成"，是RAG最原始、最简单的实现形态。

特点：

- 流程固定：用户问题 → 向量化 → 检索相关文档 → 拼接提示词 → LLM生成答案。
- 实现简单：代码量少，易于理解和快速原型验证。
- 局限性明显：
 - 检索质量完全依赖向量相似度，对专有名词、缩写等精确匹配场景效果差。
 - 没有对用户问题进行任何优化，问题表述不清时会直接影响检索质量。
 - 上下文窗口固定，无法处理长文档或多轮对话。

2 高级 RAG（Advanced RAG）

定义：在基础RAG的基础上，对检索前和检索后两个阶段进行专项优化，提升检索质量和生成效果。

特点：

🔍 检索前优化（Query Pre-processing）：

- 查询扩展（Query Expansion）：使用同义词、相关词扩展原始查询，增加召回覆盖面。
- 查询改写（Query Rewriting）：利用LLM将模糊或口语化的问题改写为更清晰、更利于检索的表述。
- 查询优化（Query Optimization）：如HyDE（假设性文档嵌入），让LLM先生成一个假设答案，再用假设答案去检索。

🔄 检索后优化（Post-retrieval）：

- 重排序（Reranking）：使用更强大的交叉编码器模型，对初步检索结果重新打分排序，优中选优。
- 上下文压缩（Context Compression）：从检索到的长文档中提取最精华的片段，减少噪音，提升生成质量。

3 模块级 RAG（Modular RAG）

定义：将RAG系统的各个功能组件（如加载器、分块器、嵌入模型、向量库、检索器、生成器等）进行**标准化、解耦和封装**，使其具备**独立开发、独立部署、独立升级**的能力。

特点：

- **高复用性**：每个模块都可以被不同的项目复用，无需重复开发。
- **兼容适配好**：可以灵活替换组件（如从OpenAI嵌入模型切换到本地HuggingFace模型），而不影响整体流程。
- **可迁移性强**：整个RAG流水线可以轻松迁移到不同的部署环境（本地、云服务器、边缘设备）。
- **便于团队协作**：不同成员可以并行开发不同的模块，通过标准化接口对接。

4 Agent RAG（Agentic RAG）

定义：将RAG从被动的问答流水线升级为拥有**自主决策能力的智能代理（Agent）**。Agent能够主动规划、使用工具、多步推理，完成复杂的任务。

特点：

- **自主规划**：Agent会分析用户问题，自主决定需要执行哪些步骤（如先搜索、再打开文档、再跳转到特定章节）。
- **工具使用**：Agent被赋予一系列工具（如搜索引擎、文档阅读器、代码执行器、API调用等），可以根据需要主动调用。
- **多步推理**：Agent不会满足于一次检索，它会在信息不足时主动发起新的检索或查询，形成"思考-行动-观察"的循环。
- **复杂问题解决**：从单纯的"问答"升级为"任务解决"，例如"帮我分析这份财报中营收下降的原因，并对比竞品同期数据"。

5 其他更复杂 RAG 探索

定义：以上四种范式之外的前沿探索方向，旨在突破传统RAG的局限性，拓展其能力边界。

代表性方向：

- **Graph RAG（知识图谱RAG）**：从文档中抽取实体和关系构建知识图谱，支持**多跳推理**（如"A公司的CEO毕业于哪所大学？"），答案更精确、可解释。
- **多模态 RAG（Multimodal RAG）**：打破纯文本限制，支持对**图像、图表、视频**等多模态信息的检索和理解，如UniversalRAG提出的"智能路由"框架。
- **终身学习 RAG（Lifelong RAG）**：使RAG系统能够持续从新的对话和反馈中学习，动态更新知识库和检索策略。
- **自我反思 RAG（Self-RAG）**：让LLM在生成答案的同时，对自己的回答进行反思和验证，发现并纠正错误。

维度对比表

比较维度	基础 RAG	高级 RAG	模块级 RAG	Agent RAG	其他复杂 RAG
核心特征	固定线性流程	检索前后双向优化	标准化、解耦、可复用	自主规划、工具使用、多步推理	图谱推理、多模态、自我反思
复杂度	★ 极低	★★★★ 中等	★★ 中低（架构设计复杂，使用简单）	★★★★★★ 很高	★★★★★ 高
检索质量	一般（依赖向量相似度）	高（多策略组合）	取决于具体模块配置	极高（多轮、多工具协同）	极高（多跳推理、多模态融合）
灵活性	低（流程固定）	中等（可配置优化策略）	高（模块可插拔）	很高（Agent自主决策）	高（针对特定场景深度定制）
可维护性	高（逻辑简单）	中等	很高（模块独立）	中等（逻辑复杂，调试难度大）	中等（依赖前沿技术栈）
适用场景	快速原型验证、简单问答	生产级问答系统、知识库问答	企业级多项目复用、多团队协作	复杂任务解决、研究分析、自动化流程	专业领域深度应用（金融、医疗、科研）
代表技术/框架	LangChain基础链	重排序模型、HyDE、查询改写	LangChain模块化设计、LlamaIndex	LangChain Agents、Mistral Agentic Search	Graph RAG、UniversalRAG、Self-RAG
代码示例参考	文档"极简RAG演示"	文档"进阶检索"章节	文档"提示词模板"中的模块化设计	文档"高阶检索"中的代理式检索	文档"高阶检索"中的知识图谱/多模态

选择建议

你的需求	推荐方案
刚入门，想快速体验RAG效果	基础 RAG → 文档中的"极简RAG演示"
准备上线产品，追求检索效果	高级 RAG → 重点优化重排序和查询改写
团队有多项目复用需求	模块级 RAG → 标准化各组件，建立内部组件库
需要处理复杂、多步骤的任务	Agent RAG → 赋予Agent自主规划和工具调用能力
领域特殊（如法律文书、科研论文）	其他复杂 RAG → 探索Graph RAG或多模态方案